

k Nearest Neighbors

discriminative
generative
model

Adapted from slides from BYU, U. of
Edinburgh and other online sources

Outline

- Intuition of kNN
- kNN algorithm
 - Choose k value *k=1, 3, 5 odd*
 - Distance measurement
- kNN for regression
 - Continuous value
- Make kNN run faster
 - K-D tree
 - Inverted list

Instance-Based Learning

$$y = f(x)$$

- Instance-based learning is often termed lazy learning, as there is typically no “transformation” of training instances into more general “statements”
- Instead, the presented training data is simply stored and, when a new query instance is encountered, a set of similar, related instances is retrieved from memory and used to classify the new query instance
- Hence, instance-based learners never form an explicit general hypothesis regarding the target function. They simply compute the classification of each new query instance as needed

no explicit model

5-NN

$\vec{x}$	y
x_1, x_2	+1
$\vdots$	$\vdots$
x_i, x_j	-1
$\vdots$	$\vdots$
x_k, x_l	+1
$\vdots$	$\vdots$
new $\vec{x}_1, \vec{x}_2$	

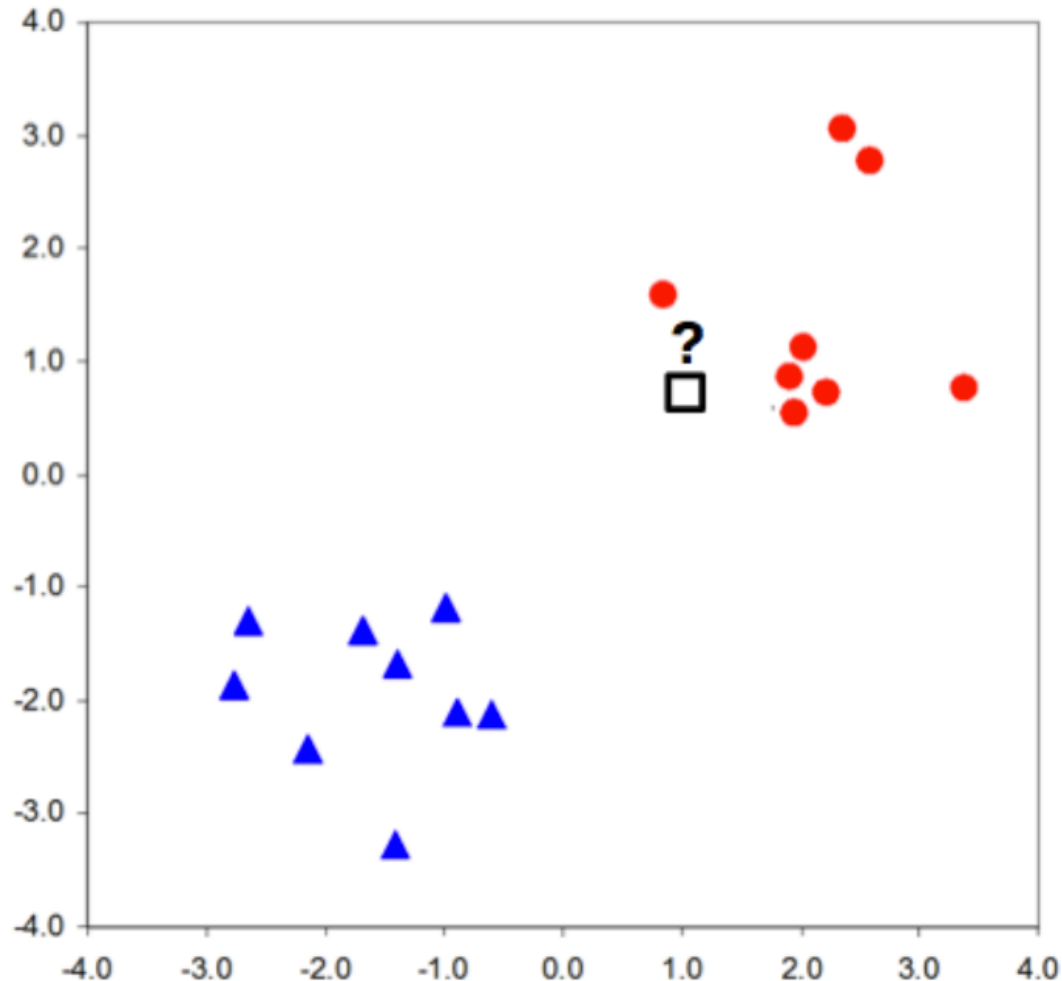
DATA

k -NN algorithm

- The simplest, most used instance-based learning algorithm is the k -NN algorithm
- k -NN assumes that all instances are points in some n -dimensional space and defines neighbors in terms of distance (usually Euclidean in $\mathbb{R}^n$ -space) $\mathbb{R}^1, \mathbb{R}^2, \mathbb{R}^3$ $\vec{x}_n$
- k is the number of neighbors considered

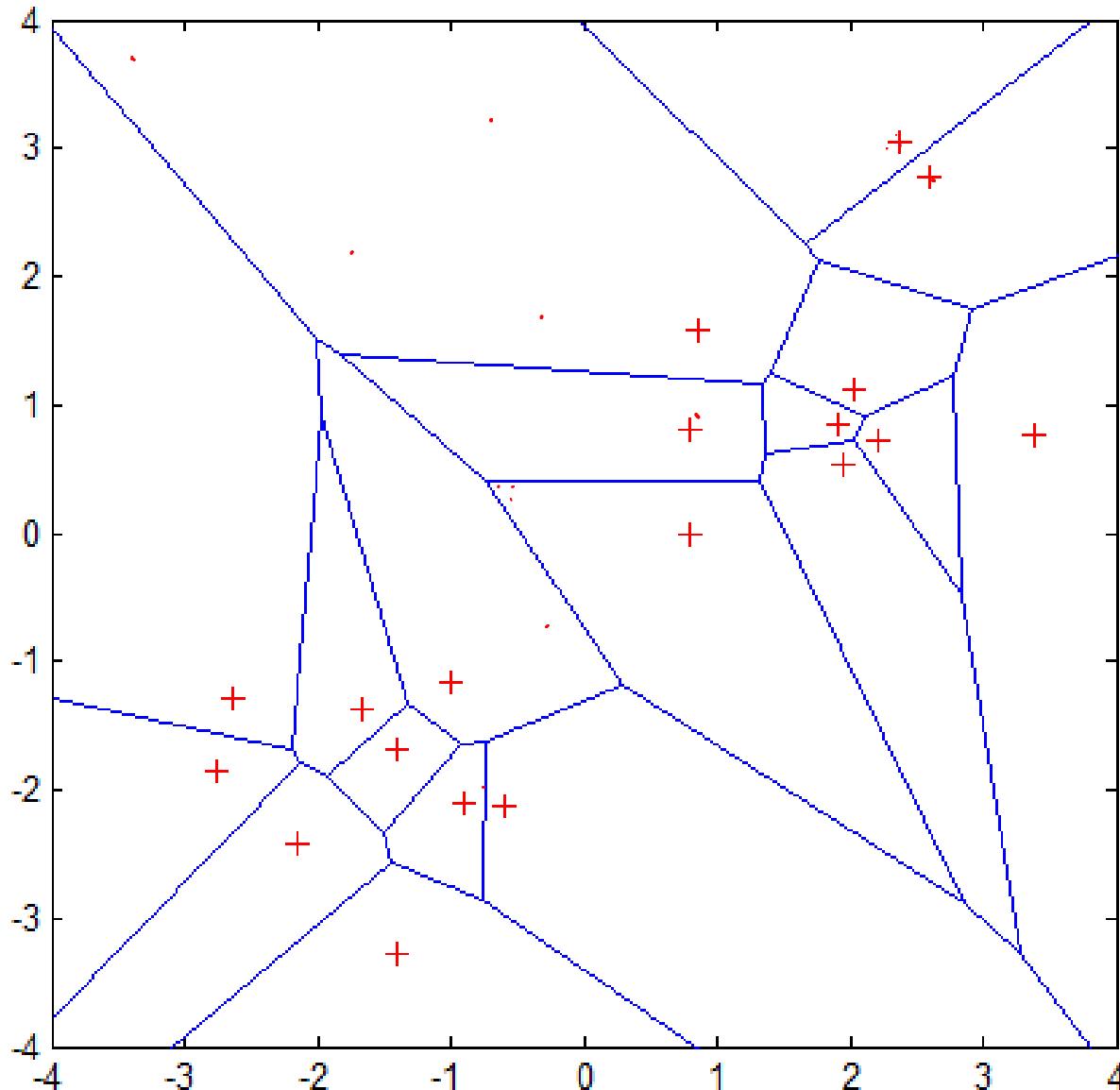
$K \rightarrow D$

Intuition for kNN



- set of points (x,y)
 - two classes
- is the box red or blue
- how did you do it
 - use Bayes rule?
 - a decision tree?
 - fit a hyperplane?
- nearby points are red
 - use this as a basis for a learning algorithm

Graphic Depiction



Properties:

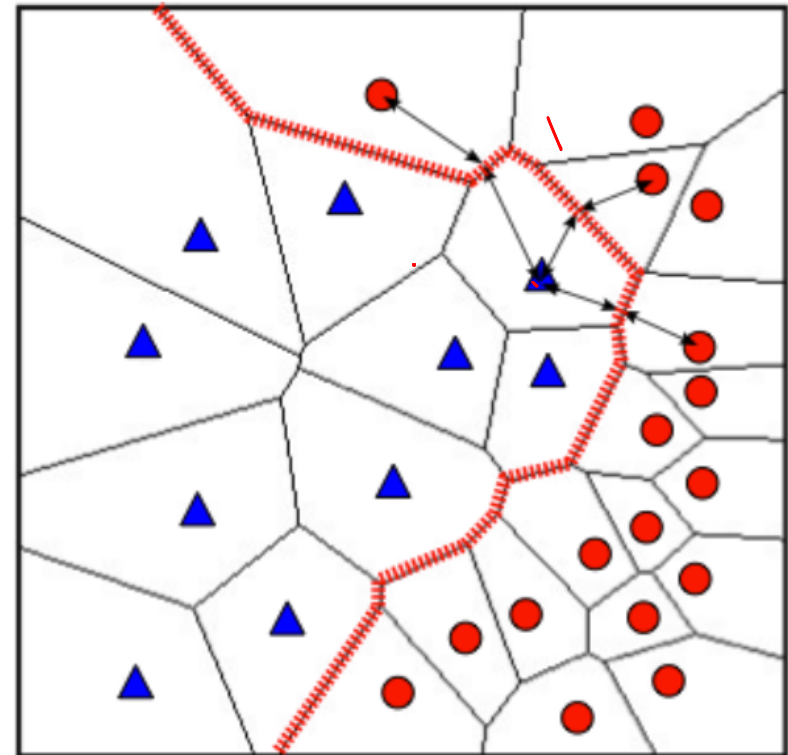
- 1) All possible points within a sample's Voronoi cell are the nearest neighboring points for that sample
- 2) For any sample, the nearest sample is determined by the closest Voronoi cell edge

Basic Idea

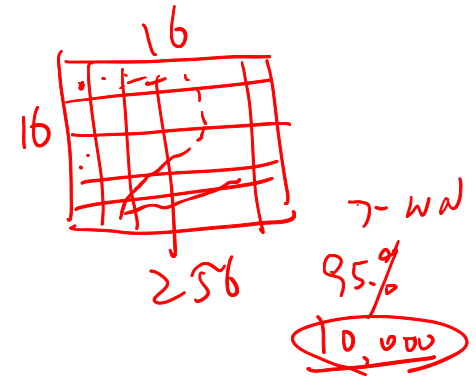
- Using the second property, the k -NN classification rule is to assign to a test sample the majority category label of its k nearest training samples
- In practice, k is usually chosen to be odd, so as to avoid ties
- The $k = 1$ rule is generally called the nearest-neighbor classification rule

Nearest Neighbor Classifier

- Use the intuition to classify a new point x :
 - find the most similar training example x'
 - predict its class y'
- Voronoi tessellation
 - partitions space into regions
 - boundary: points at same distance from two different training examples
- classification boundary
 - non-linear, reflects classes well
 - compare to regression, DTrees
 - impressive for simple method



k-NN Algorithm



- Given:
 - training examples $\{x_i, y_i\}$
 - x_i ... attribute-value representation of examples
 - y_i ... class label: {ham,spam}, digit {0,1,...9} etc. *10 classes*
 - testing point x that we want to classify
- Algorithm:
 - compute distance $D(x, x_i)$ to every training example x_i
 - select k closest instances $x_{i1} \dots x_{ik}$ and their labels $y_{i1} \dots y_{ik}$
 - output the class y^* which is most frequent in $y_{i1} \dots y_{ik}$

$$\sum_{i=1}^k y_i > 0$$

Handwritten red notes: $\frac{+1}{-1}$

k -NN Algorithm

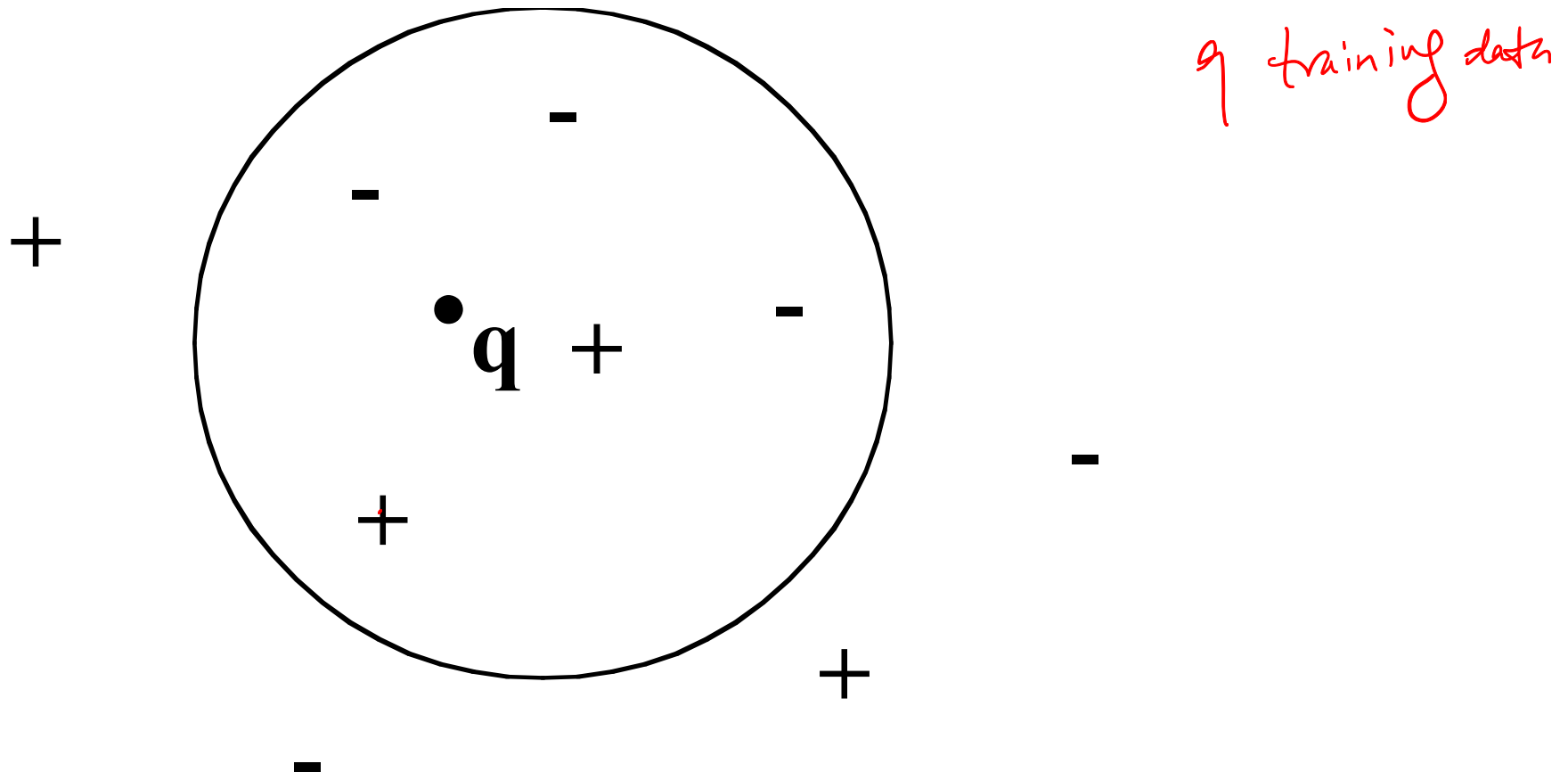
- For each training instance $t=(x, f(x))$
 - Add t to the set $Tr_instances$
- Given a query instance q to be classified
 - Let $x_1, \dots, x_k$ be the k training instances in $Tr_instances$ nearest to q
 - Return

$$\hat{f}(q) = \arg \max_{v \in V} \sum_{i=1}^k \delta(v, f(x_i))$$

majority

- Where V is the finite set of target class values, and $\delta(a,b)=1$ if $a=b$, and 0 otherwise (Kronecker function)
- Intuitively, the k -NN algorithm assigns to each new query instance the majority class among its k nearest neighbors

Simple Illustration

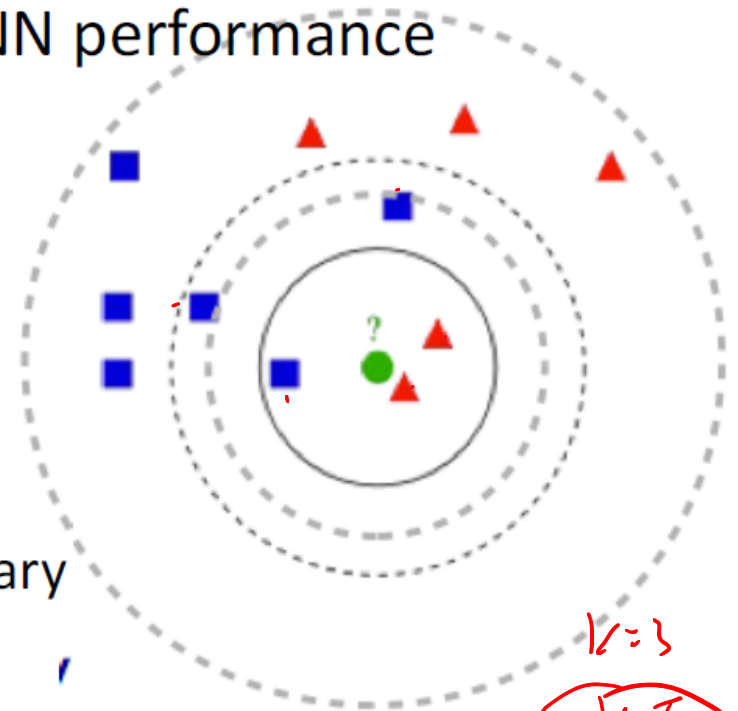


q is + under 1-NN, but - under 5-NN

Choose the value of k

- Value of k has strong effect on kNN performance

- large value $\rightarrow$ everything classified as the most probable class: $P(y)$
- small value $\rightarrow$ highly variable, unstable decision boundaries
 - small changes to training set $\rightarrow$ large changes in classification
- affects “smoothness” of the boundary

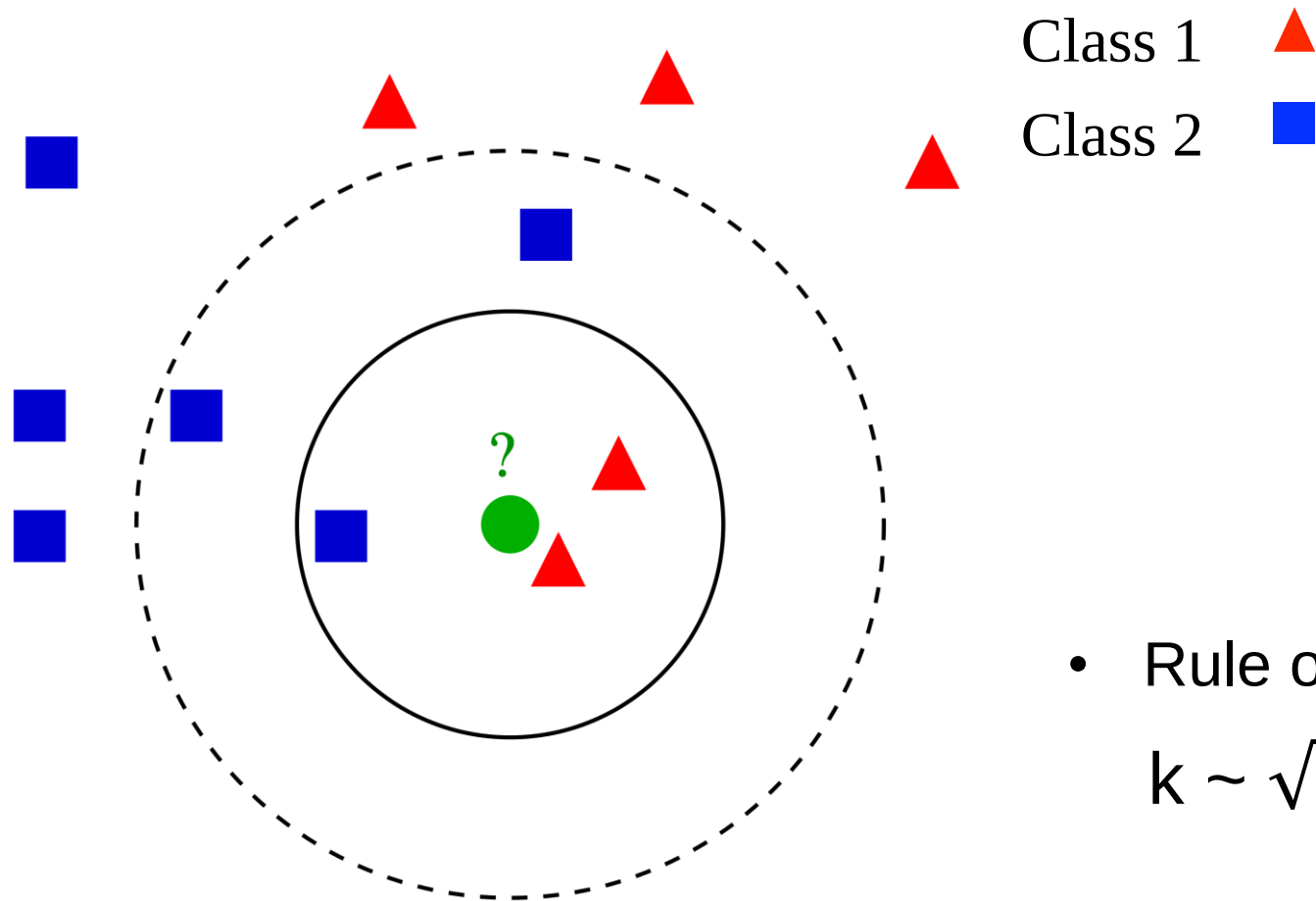


- Selecting the value of k

- set aside a portion of the training data (validation set)
- vary k , observe training $\rightarrow$ validation error *Smallest*
- pick k that gives best generalization performance

$k=3$
 $k=5$
 $k=7$
 $k=9$

Choose the value of k



- Rule of thumb:
 $k \sim \sqrt{N}$

Distance Measurement

- Key component of the kNN algorithm
 - defines which examples are similar & which aren't
 - can have strong effect on performance
- Euclidian (numeric attributes): $D(x, x') = \sqrt{\sum_d |x_d - x'_d|^2}$
 - symmetric, spherical, treats all dimensions equally
 - sensitive to extreme differences in single attribute
 - behaves like a “soft” logical OR
- Hamming (categorical attributes): $D(x, x') = \sum_d 1_{x_d \neq x'_d}$
 - number of attributes where x, x' differ

Distance Metrics

Minkowsky:

$$D(x, y) = \left(\sum_{i=1}^m |x_i - y_i|^r \right)^{1/r}$$

Euclidean:

$$D(x, y) = \sqrt{\sum_{i=1}^m (x_i - y_i)^2}$$

Manhattan / city-block:

$$D(x, y) = \sum_{i=1}^m |x_i - y_i|$$

Camberra:

$$D(x, y) = \sum_{i=1}^m \frac{|x_i - y_i|}{|x_i + y_i|}$$

Chebychev:

$$D(x, y) = \max_{i=1}^m |x_i - y_i|$$

Quadratic:

$$D(x, y) = (x - y)^T Q (x - y) = \sum_{j=1}^m \left(\sum_{i=1}^m (x_i - y_i) q_{ji} \right) (x_j - y_j)$$

Q is a problem-specific positive definite $m \times m$ weight matrix

Mahalanobis:

$$D(x, y) = [\det V]^{1/n} (x - y)^T V^{-1} (x - y)$$

Anti-sticks

V is the covariance matrix of $A_1..A_m$, and A_j is the vector of values for attribute j occurring in the training set instances $1..n$.

Correlation:

$$D(x, y) = \frac{\sum_{i=1}^m (x_i - \bar{x}_i)(y_i - \bar{y}_i)}{\sqrt{\sum_{i=1}^m (x_i - \bar{x}_i)^2 \sum_{i=1}^m (y_i - \bar{y}_i)^2}}$$

$\bar{x}_i = \bar{y}_i$ and is the average value for attribute i occurring in the training set.

Chi-square:

$$D(x, y) = \sum_{i=1}^m \frac{1}{sum_i} \left(\frac{x_i}{size_x} - \frac{y_i}{size_y} \right)^2$$

sum_i is the sum of all values for attribute i occurring in the training set, and $size_x$ is the sum of all values in the vector x .

Kendall's Rank Correlation:

$$D(x, y) = 1 - \frac{2}{n(n-1)} \sum_{i=1}^m \sum_{j=1}^{i-1} \text{sign}(x_i - x_j) \text{sign}(y_i - y_j)$$

$\text{sign}(x) = -1, 0$ or 1 if $x < 0$, $x = 0$, or $x > 0$, respectively.

Figure 1. Equations of selected distance functions.
(x and y are vectors of m attribute values).

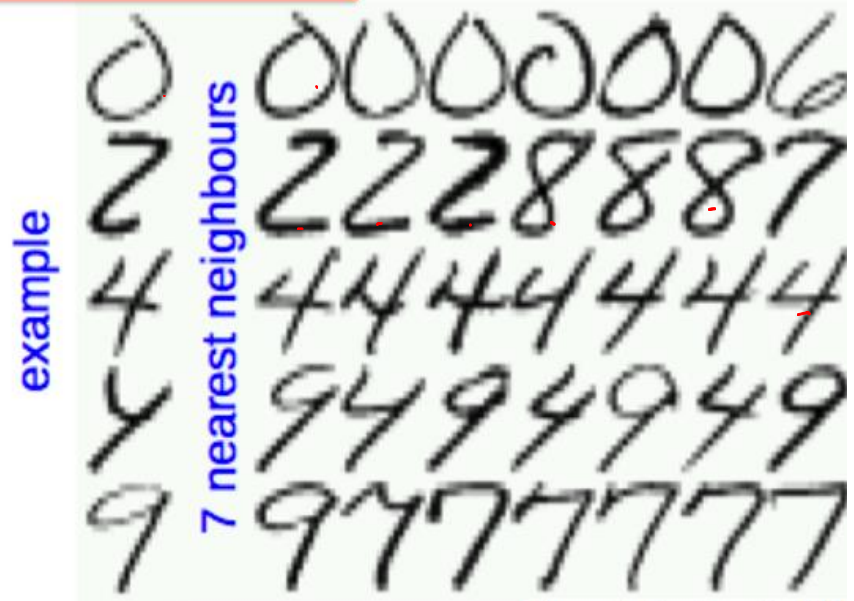
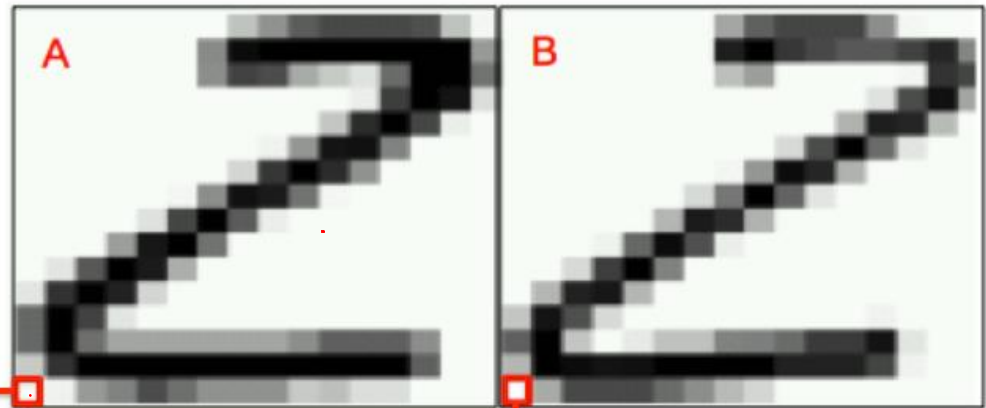
kNN Example: Hand-Written Digit

- 16x16 bitmaps
- 8-bit grayscale
- Euclidian distance

– over raw pixels

$$D(A,B) = \sqrt{\sum_r \sum_c (A_{r,c} - B_{r,c})^2}$$

- Accuracy:
 - 7-NN ~ 95.2%
 - SVM ~ 95.8%
 - humans ~ 97.5%

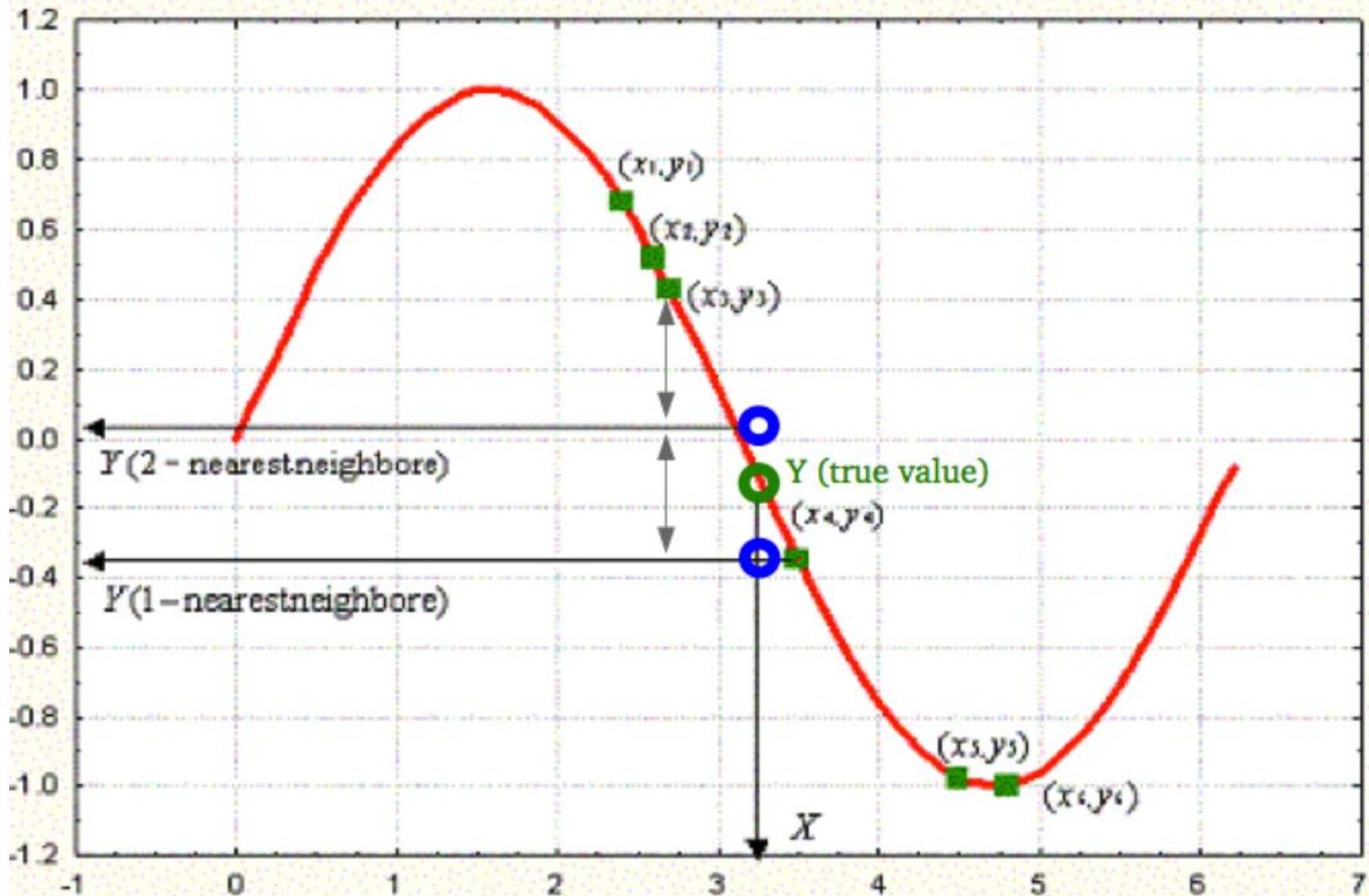


k-NN Regression

- Given:
 - training examples $\{x_i, y_i\}$
 - x_i ... attribute-value representation of examples
 - y_i ... real-valued target (profit, rating on YouTube, etc)
 - testing point x that we want to predict the target
- Algorithm:
 - compute distance $D(x, x_i)$ to every training example x_i
 - select k closest instances $x_{i1} \dots x_{ik}$ and their labels $y_{i1} \dots y_{ik}$
 - output the mean of $y_{i1} \dots y_{ik}$:

$$\hat{y} = f(x) = \frac{1}{k} \sum_{j=1}^k y_{i_j}$$

K-NN Regression in 1D



Predicting Continuous Values

- Replace $\hat{f}(q) = \arg \max_{v \in V} \sum_{i=1}^k w_i d(v, f(x_i))$
by:

$$\hat{f}(q) = \frac{\sum_{i=1}^k w_i f(x_i)}{\sum_{i=1}^k w_i}$$

- Note: unweighted corresponds to $w_i=1$ for all i

kNN Practical Issues

- Resolving ties:
 - equal number of positive/negative neighbours
 - use odd k (doesn't solve multi-class)
 - breaking ties:
 - random: flip a coin to decide positive / negative
 - prior: pick class with greater prior
 - nearest: use 1-nn classifier to decide
- Missing values
 - have to “fill in”, otherwise can't compute distance
 - key concern: should affect distance as little as possible
 - reasonable choice: average value across entire dataset

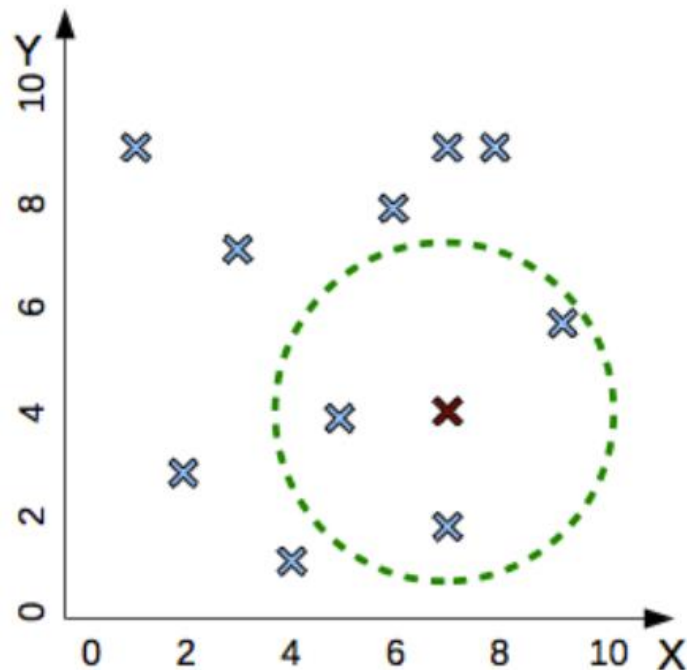
kNN Pros and Cons

- Almost no assumptions about the data
 - smoothness: nearby regions of space $\rightarrow$ same class
 - assumptions implied by distance function (only locally!)
 - non-parametric approach: “let the data speak for itself”
 - nothing to infer from the data, except k and possibly $D()$
 - easy to update in online setting: just add new item to training set
- Computationally expensive:
 - space: need to store all training examples
 - time: need to compute distance to all examples: $O(ND)$
 - N ... number of training examples, D ... cost of computing distance
 - N grows $\rightarrow$ system will become slower and slower
 - expense is at *testing*, not *training* time (bad)

kNN Complexity

$O(Nd)$

What you see



What algorithm sees

- Training set:

$\{(1,9), (2,3), (4,1),$
 $(3,7), (5,4), (6,8),$
 $(7,2), (8,8), (7,9), (9,6)\}$

- Testing instance:

$(7,4)$

- Nearest neighbors?

compare one-by-one
to each training instance

- n comparisons
- each takes d operations

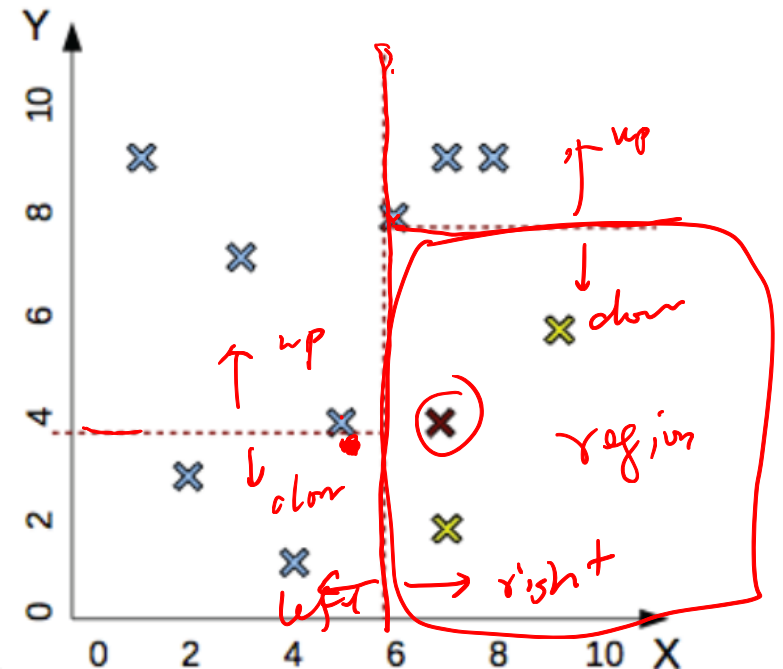
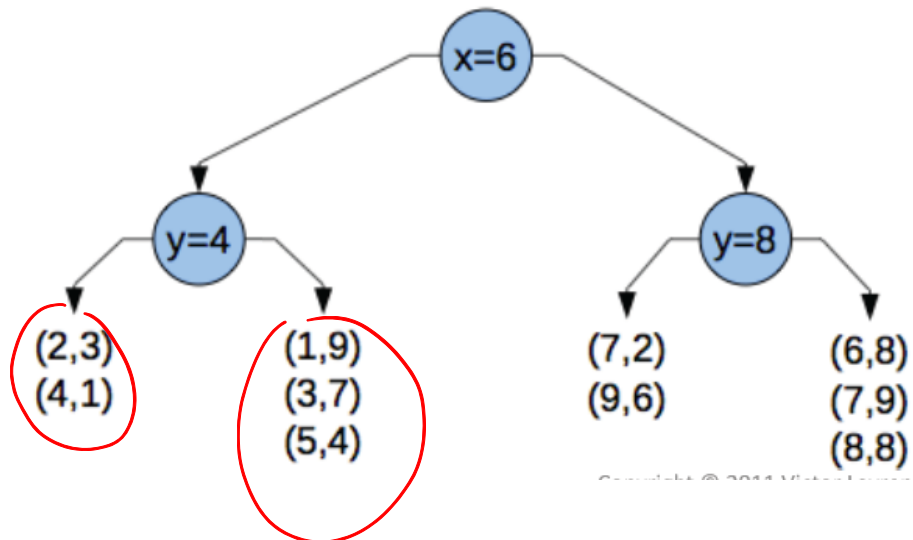
Making kNN Faster

- Training: $O(1)$, but testing: $O(ND)$
- Reduce D : dimensionality reduction
 - simple feature selection, other methods $O(D^3)$
- Reduce N : don't compare to **all** training examples
 - idea: quickly identify $n \ll N$ potential near neighbors
 - compare only to those, pick k nearest neighbors $\rightarrow O(nD)$ time
 - **K-D trees**: low-dimensional, numeric data
 - $O(D \log N)$, only works when $D \ll N$, inexact: may miss neighbors nearest accuracy lost
 - **inverted lists**: high-dimensional, discrete data
 - $O(dn)$ where $d \ll D$, $n \ll N$, only for sparse data (e.g. text), exact
 - **fingerprinting**: high-d, sparse or dense
 - $O(Dn)$, $n \ll N$... bits in fingerprint, inexact: may miss near neighbors

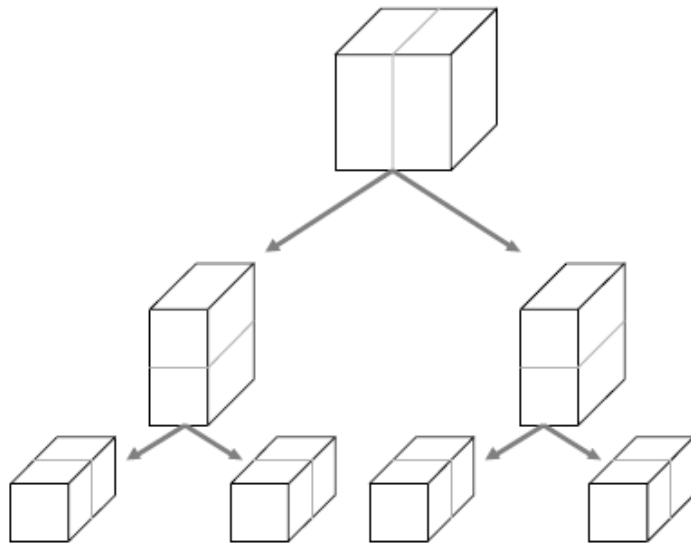
$P \subset A$

K-D Tree *fast*

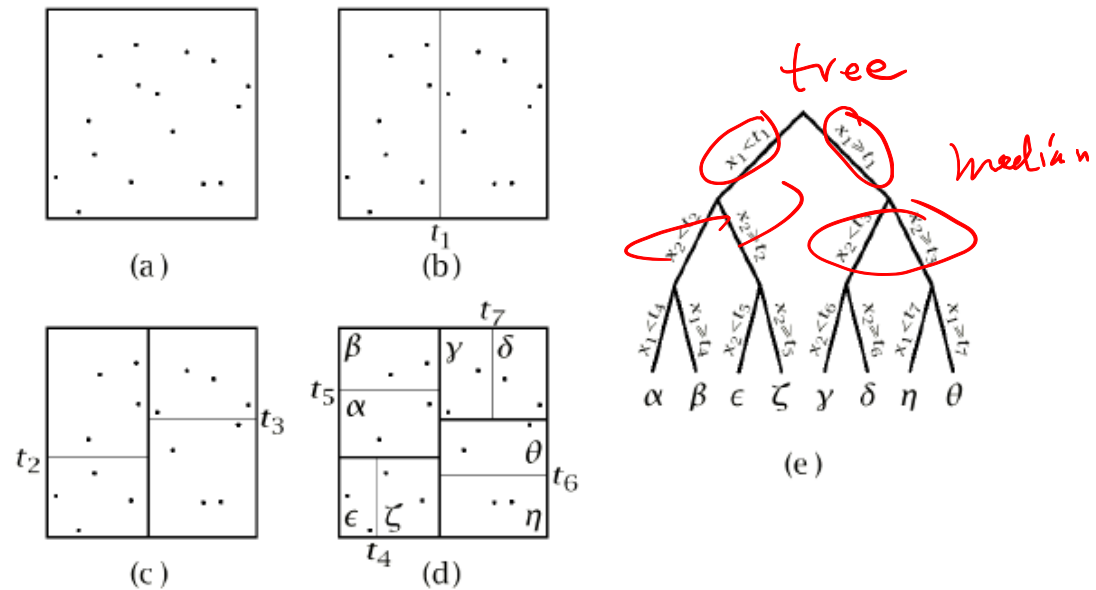
- Building a K-D tree from training data:
 - $\{(1,9), (2,3), (4,1), (3,7), (5,4), (6,8), (7,2), (8,8), (7,9), (9,6)\}$
 - pick random dimension, find median, split data, repeat
- Find NNs for new point (7,4)
 - find region containing (7,4)
 - compare to all points in region



K-D Tree Structure



3D structure



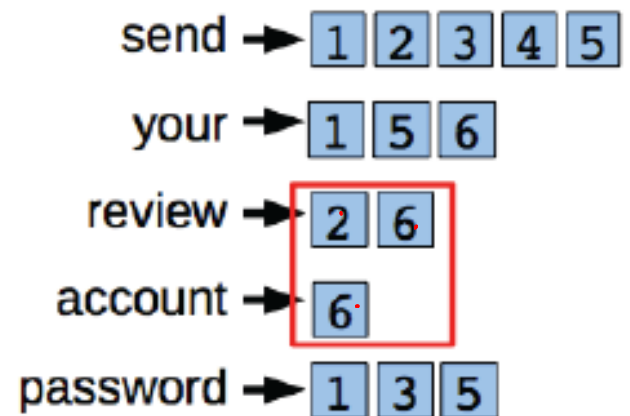
2D structure

Inverted List

- Data structure used by search engines (Google, etc)
 - list all training examples that contain particular attribute
 - assumption: most attribute values are zero (sparseness)
- Given a new testing example:
 - merge inverted lists for nonzero attributes in new example
 - $O(dn)$: d ... nonzero attributes, n ... avg. length of inverted list

D1: "send your password" spam
D2: "send us review" ham
D3: "send us password" spam
D4: "send us details" ham
D5: "send your password" spam
D6: "review your account" ham

new email: "account review"



Some Remarks

- k -NN works well on many practical problems and is fairly noise tolerant (depending on the value of k)
- k -NN is subject to the curse of dimensionality (i.e., presence of many irrelevant attributes) $O(ND)$
- k -NN needs adequate distance measure
- k -NN relies on efficient indexing

k -D tree

How is kNN Incremental?

- All training instances are stored
- Model consists of the set of training instances
- Adding a new training instance only affects the computation of neighbors, which is done at execution time (i.e., lazily)

Summary of kNN

- Key idea: nearby points -> same class
 - Important to select good distance function
- Can be used for classification and regression
- Simple, nonlinear, asymptotically optimal
 - Does not make assumptions about the data
 - “let the data data speak for itself”
- Select k that yields the best performance
- Naïve implement is slow for large dataset
 - Use K-D tree or inverted list

Evaluation
dataset